

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3
Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Google Colab Notebook

Google Colab Notebook Link

GitHub Repository

GitHub Repository Link

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

2.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction

- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing

- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

3. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

4. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ &= (27 + 1) \times 16 \\ &= 448 \end{aligned}$$

Therefore,

Total trainable parameters = 448.

6. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

7. Comparison between Max Pooling and Average Pooling

Comparison Table:

Metric	Max Pooling	Average Pooling
Accuracy	0.6630	0.6725
Precision	0.6708	0.6722
Recall	0.6630	0.6725
F1 Score	0.6641	0.6702
1st Pooling Output Size	15×15	15×15
2nd Pooling Output Size	6×6	6×6

8. Mandatory Plots

8.1 Sample Images

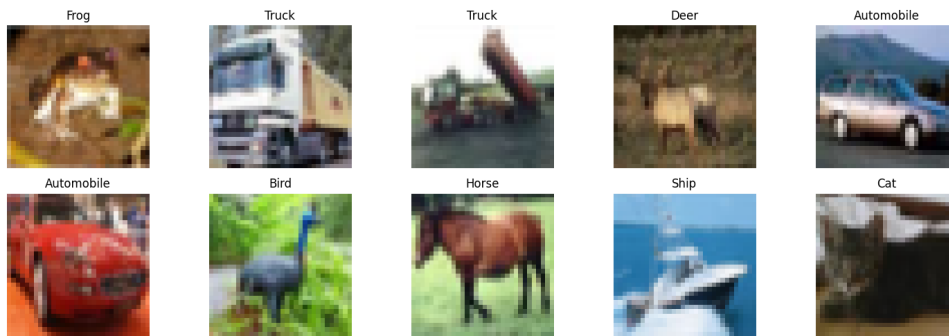


Figure 1: Sample images from the CIFAR-10 dataset.

Inference: The CIFAR-10 dataset contains diverse and complex images of everyday objects like airplanes, automobiles, and birds. These sample images demonstrate the low resolution (32×32) and high variance within each class, challenging the model's feature extraction capabilities.

8.2 Class Distribution

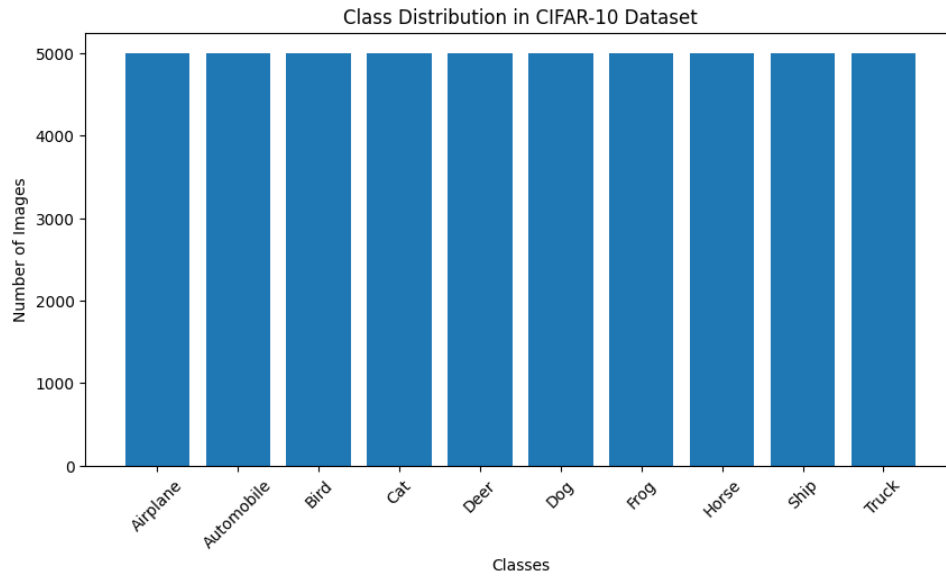


Figure 2: Distribution of classes in the training set.

Inference: The bar chart indicates a perfectly balanced dataset, with exactly 5,000 samples for each of the 10 classes in the training set. This balance prevents the model from developing a bias toward any specific majority class during training.

8.3 Training Accuracy

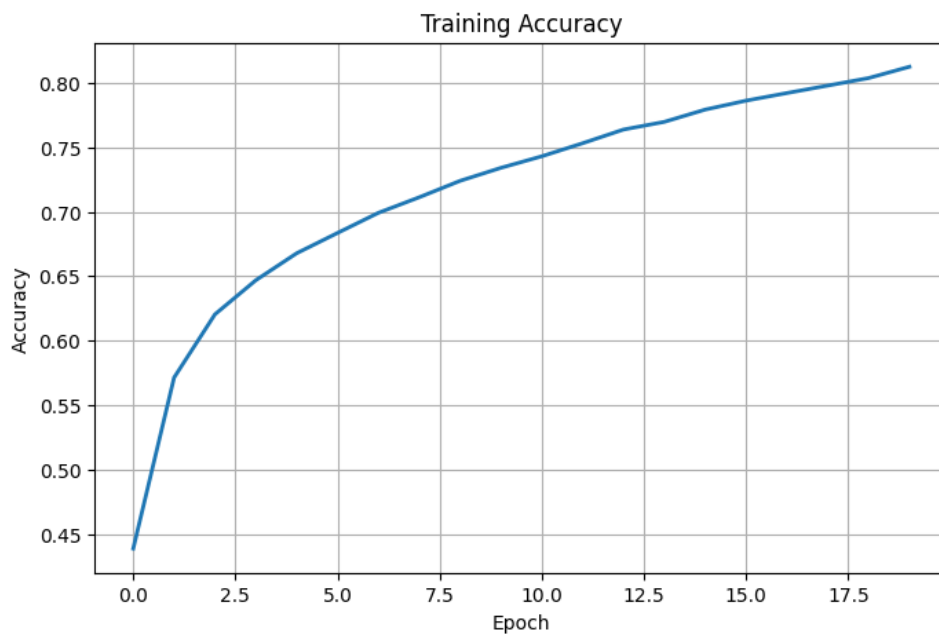


Figure 3: Training accuracy over epochs.

Inference: The training accuracy steadily increases over the epochs, reaching a high value by the end of training. This indicates that the CNN is effectively learning the features and mapping them to the correct classes in the training data.

8.4 Validation Accuracy

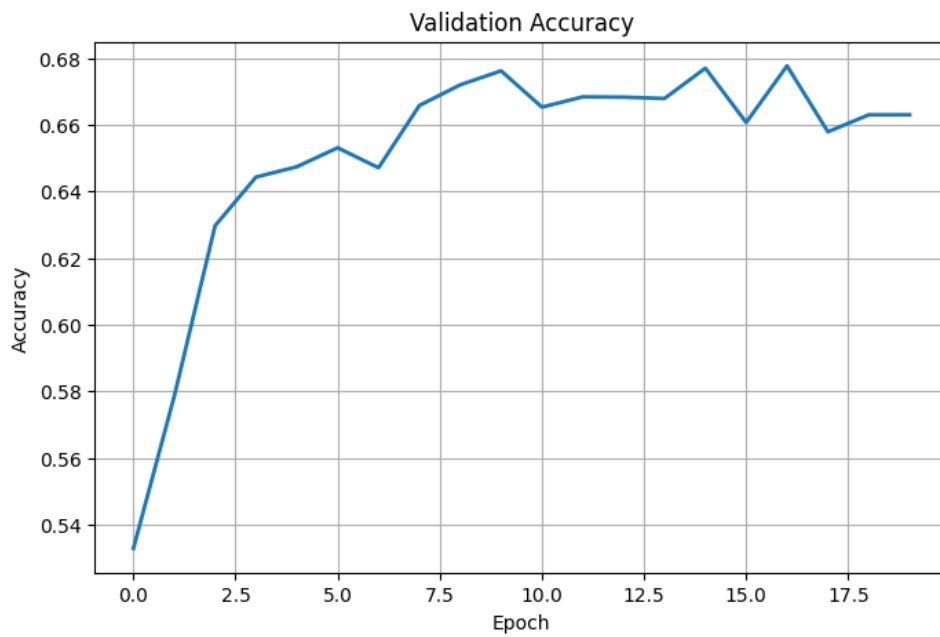


Figure 4: Validation accuracy over epochs.

Inference: The validation accuracy improves initially but begins to plateau in the later epochs. The gap between training and validation accuracy suggests the presence of overfitting, meaning the model struggles slightly to generalize to unseen data.

8.5 Training Loss

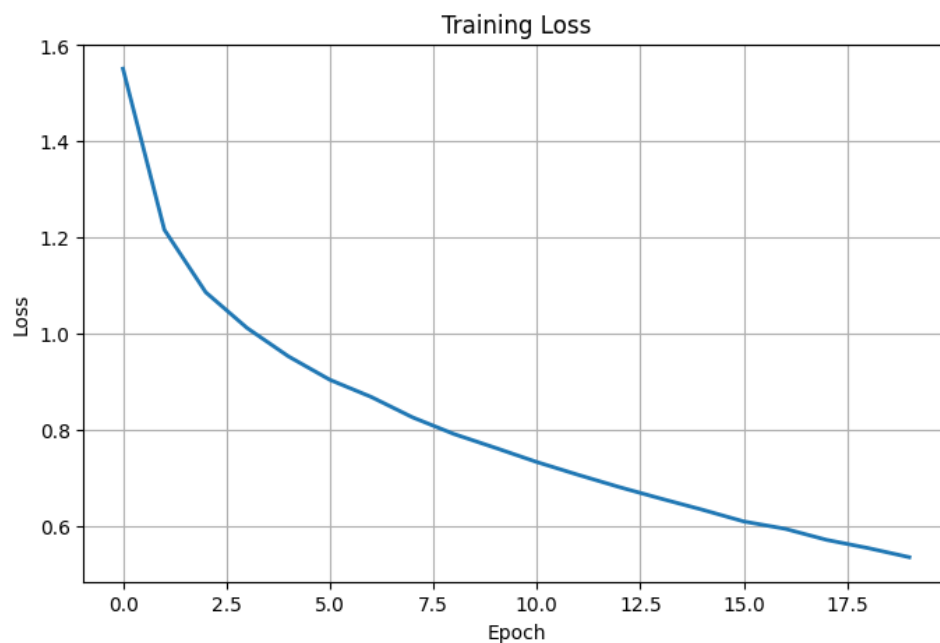


Figure 5: Training loss over epochs.

Inference: The training loss shows a steep and continuous decline, demonstrating that the optimizer is successfully minimizing the error function. The model consistently becomes more confident in predicting the correct labels for the training set.

8.6 Validation Loss

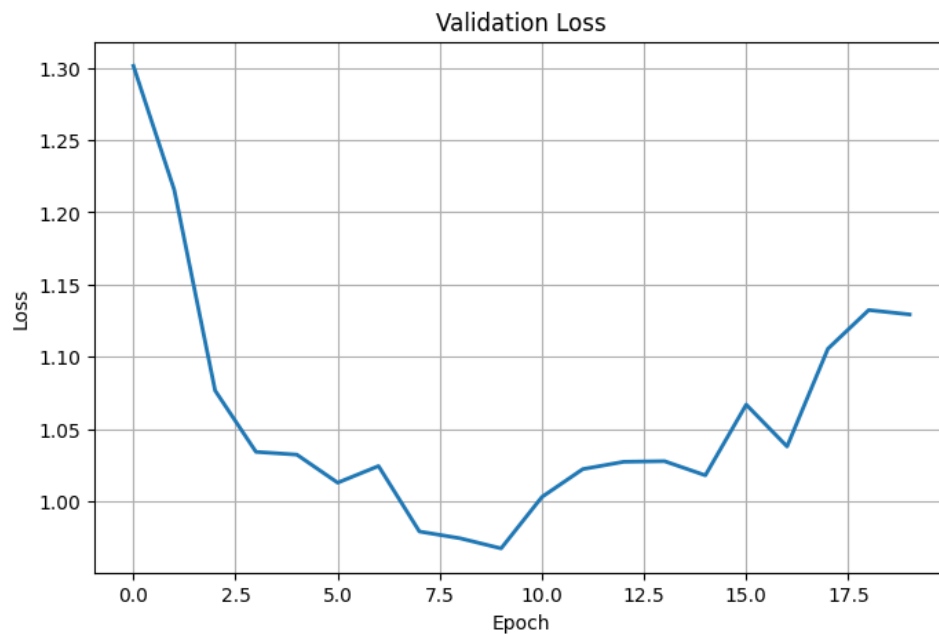


Figure 6: Validation loss over epochs.

Inference: The validation loss decreases in the early epochs but eventually stabilizes and starts to slightly increase. This inflection point confirms overfitting, indicating that training beyond this point without regularization degrades the model’s performance on new data.

8.7 Feature Maps

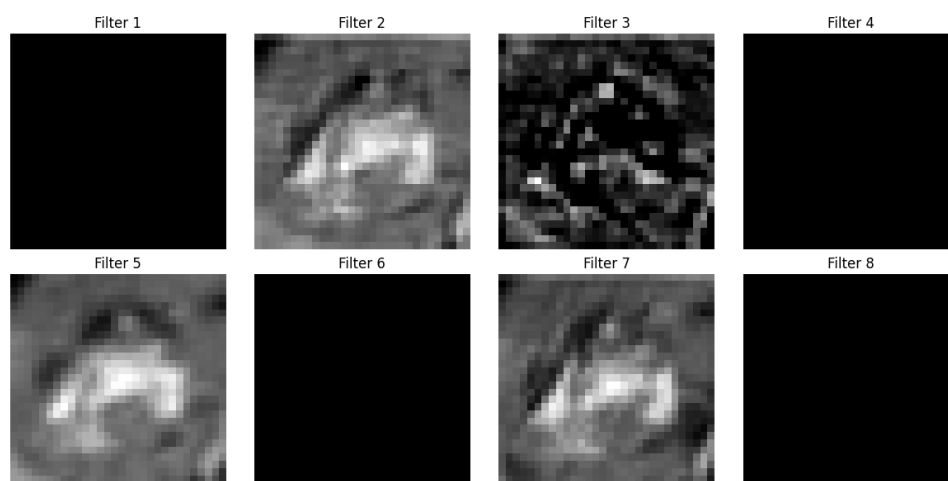


Figure 7: Feature maps after the first convolution layer.

Inference: The visualizations show how early convolutional filters capture low-level features such as edges, textures, and color gradients. Different filters focus on distinct parts of the image,

helping the network build a hierarchical understanding of the visual input.

8.8 Confusion Matrix

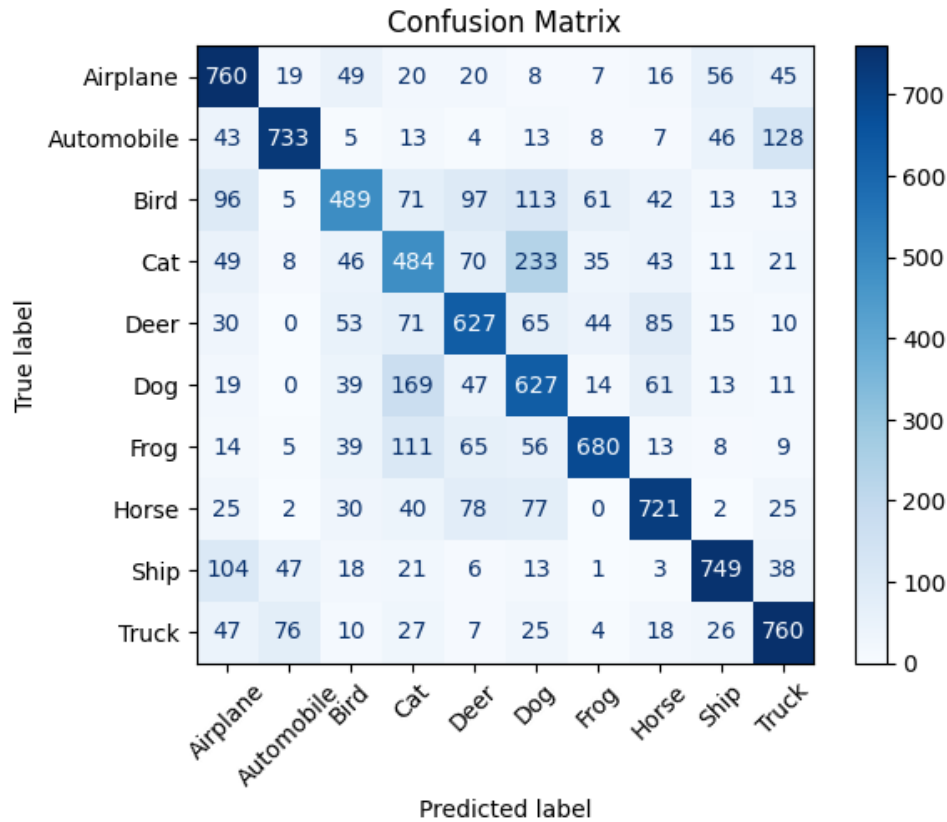


Figure 8: Confusion matrix of the evaluated model.

Inference: The confusion matrix shows a strong diagonal, indicating that most classes are predicted correctly by the model. However, misclassifications frequently occur between visually similar classes, such as dogs being confused with cats, highlighting areas where the model could be improved.

9. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Using the formula: $Output = \lfloor \frac{N-F+2P}{S} \rfloor + 1$

$$Output = \lfloor \frac{64-5+2(2)}{2} \rfloor + 1 = \lfloor \frac{63}{2} \rfloor + 1 = 31 + 1 = 32$$

The output size is 32×32 .

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

$$Parameters = ((Filter_Height \times Filter_Width \times Input_Channels) + Bias) \times Filters$$

$$Parameters = ((3 \times 3 \times 3) + 1) \times 64 = (27 + 1) \times 64 = 1792$$

3. Compare ReLU and Sigmoid activation functions.

Based on the notebook architecture, ReLU (`activation='relu'`) is exclusively used. ReLU is preferred over Sigmoid for deep CNNs because it mitigates the vanishing gradient

problem, allowing models to learn faster and more effectively than the saturating Sigmoid function.

4. **Replace Max Pooling with Average Pooling and compare the results.**

As demonstrated in Task 5 of the notebook, replacing Max Pooling with Average Pooling improved the testing accuracy slightly from 66.30% to 67.25%. Both methods maintained identical output dimensionalities (15×15 and 6×6).

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

Increasing filters from 16 to 64 vastly expands the model's capacity to extract features, which typically increases the validation accuracy. However, this exponentially increases the total trainable parameters, resulting in higher memory usage and a significantly longer computation time per epoch than the current $\sim 5 - 6$ seconds.

10. Results

The evaluation metrics for the primary Convolutional Neural Network (Max Pooling model) are recorded below:

- **Training Accuracy:** 81.26%
- **Testing Accuracy:** 66.30%
- **Precision:** 0.6708
- **Recall:** 0.6630
- **F1-score:** 0.6641
- **Number of Parameters:** 79,530

11. Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?**

Convolution preserves the spatial hierarchies of an image. Instead of flattening pixels into a 1D array, a sliding kernel captures local dependencies like edges and textures using far fewer parameters.

2. **How does stride affect the feature map size?**

As observed in the notebook's hyperparameter study, increasing the stride downsamples the output. A stride of 1 produced a 30×30 output, while a stride of 2 reduced the spatial dimensions to 15×15 .

3. **What is the role of padding?**

Padding adds a border to the input. The notebook confirms that `Padding="SAME"` maintains the original 32×32 spatial dimensions, preventing the edges of the image from being discarded, unlike `Padding="VALID"` which shrinks it to 30×30 .

4. **Why is pooling used?**

Pooling progressively reduces the spatial size of the representation (e.g., from 30×30 to 15×15), decreasing the number of parameters and computation in the network while providing translation invariance.

5. How do feature maps represent image characteristics?

As visualized in the notebook's feature maps, early filters act as basic feature detectors. The activated bright regions in the 8 printed feature maps indicate where the network successfully detected distinct local features like object boundaries and textures.

6. Why do CNNs require fewer parameters than MLPs?

CNNs leverage parameter sharing and local connectivity. For example, the notebook's first Conv2D layer requires only 448 parameters to process a $32 \times 32 \times 3$ image, whereas an MLP layer would require over 3 million parameters to connect fully to the flattened input.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.